Single Linear Regressions

```
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np
```

```
dataset=pd.read_csv("/content/exp-1_train.csv")
dataset
```

	x	y
0	24.0	21.549452
1	50.0	47.464463
2	15.0	17.218656
3	38.0	36.586398
4	87.0	87.288984
...	...	...
695	58.0	58.595006
696	93.0	94.625094
697	82.0	88.603770
698	66.0	63.648685
699	97.0	94.975266

700 rows × 2 columns

Next steps: Generate code with dataset New interactive sheet

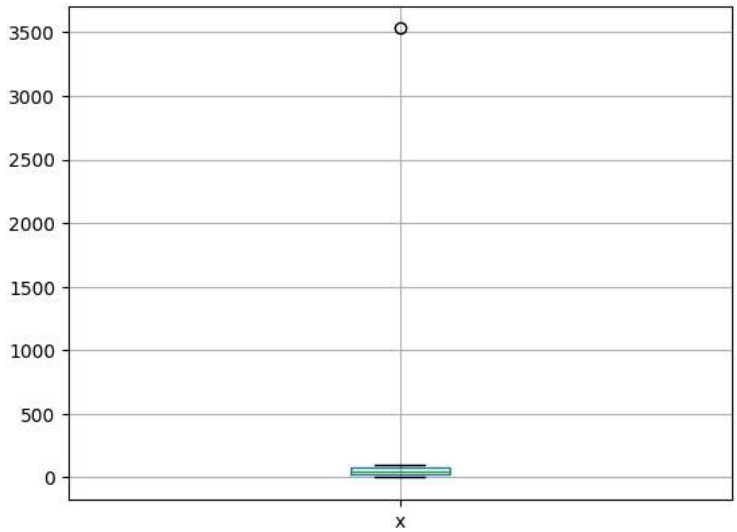
```
dataset.describe()
```

	x	y
count	700.000000	699.000000
mean	54.985939	49.939869
std	134.681703	29.109217
min	0.000000	-3.839981
25%	25.000000	24.929968
50%	49.000000	48.973020
75%	75.000000	74.929911
max	3530.157369	108.871618

```
x=dataset.iloc[0:700,0:1]
y=dataset.iloc[0:700,1:2]
```

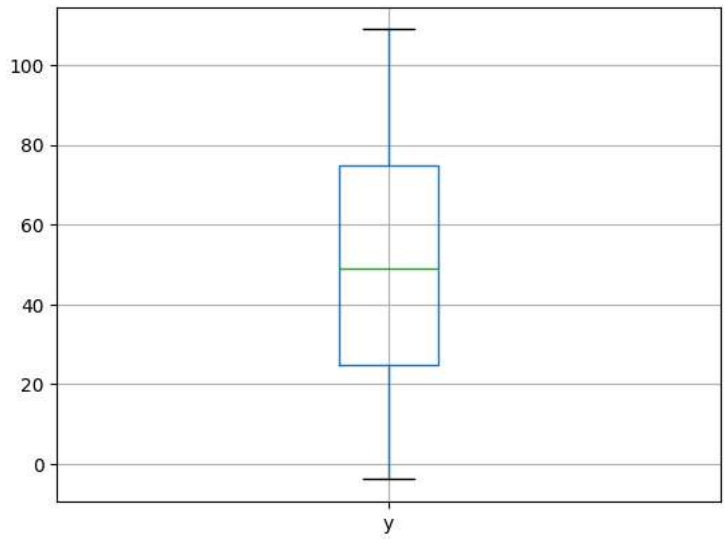
```
x.boxplot()
```

```
<Axes: >
```



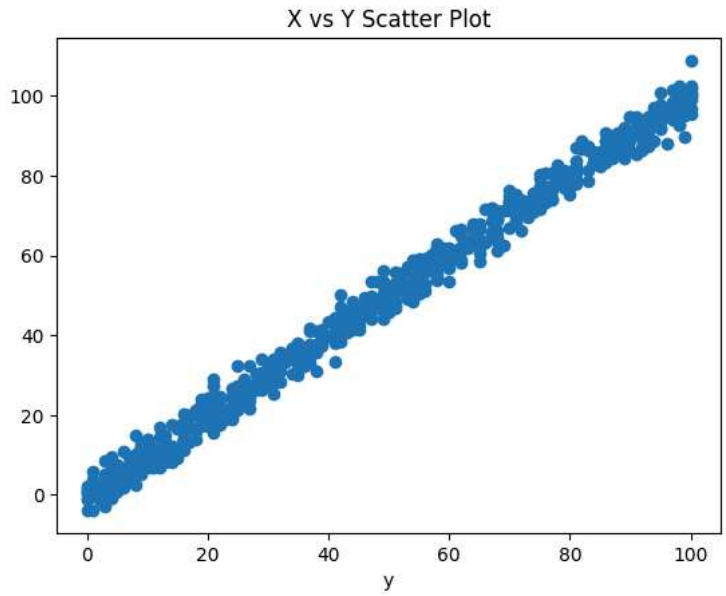

```
y.boxplot()
```

```
<Axes: >
```




```
#plot the scatter plot
plt.scatter(x,y)
plt.xlabel('x')
plt.xlabel('y')
plt.title("X vs Y Scatter Plot")
```

```
Text(0.5, 1.0, 'X vs Y Scatter Plot')
```




```
#Linear Regression
def hypothesis(theta_array,x):
  return theta_array[0]+theta_array[1]*x
```

```
def cost_function(theta_array,x,y,m):
  error=0
  for i in range(m):
    error=error+((theta_array[0]+theta_array[1]*x)-y[i])**2
  return error/(2*m)
```

```
def gradint_descent(theta_array,x,y,m,alpha):
  summation_0=0
  summation_1=0

  for i in range(m):
    summation_0+=(theta_array[0]+theta_array[1]*x[i])-y[i]
    summation_1+=x[i]*((theta_array[0]+theta_array[1]*x[i])-y[i])

  new_theta0=theta_array[0] - (alpha/m) * summation_0
  new_theta1=theta_array[1] - (alpha/m) * summation_1
  updated_new_theta=[new_theta0,new_theta1]
  return updated_new_theta
```

```
def training(x,y,alpha,iters):
  theta0=0
  theta1=0
  cost_values=[]
  theta_array=[theta0,theta1]
  m=x.size

  for i in range(iters):
    theta_array=gradint_descent(theta_array,x,y,m,alpha)
    cost_values.append(cost_function(theta_array,x,y,m))
    y_predict=x*theta_array[1]+theta_array[0]
    plt.plot(y_predict,x,'r')
    plt.scatter(x,y)
    plt.show()

  return cost_values,theta_array
```

```
#Fidding the input data
training_data=dataset.dropna()
```

```
training_data.shape
```

```
(699, 2)
```

```
x_value=training_data['x']
y_value=training_data['y']
```

```
type(x_value)
```

```
pandas.core.series.Series
def __init__(data=None, index=None, dtype: Dtype | None=None, name=None, copy: bool | None=None,
fastpath: bool | lib.NoDefault=lib.no_default) -> None
    RangeIndex (0, 1, 2, ..., n) if not provided. If data is dict-like
    and index is None, then the keys in the data are used as the index. If the
    index is not None, the resulting Series is reindexed with the index values.
dtype : str, numpy.dtype, or ExtensionDtype, optional
    Data type for the output Series. If not specified, this will be
    inferred from `data`.
    See the :ref:`user guide <basics.dtypes>` for more usages.
```

```
x_value=x_value.values.reshape(x_value.size)
y_value=y_value.values.reshape(y_value.size)
```

```
type(x_value)
```

```
numpy.ndarray
```

```
alpha=0.0001
# alpha=100
iters=50
cost_values,theta_array=training(x_value,y_value,alpha,iters)
```